



Fakultät Informatik

**Entwicklung eines VHDL-basierten Applikationsbeispiels
zur Ablösung des FM350-2 Moduls durch das TM FAST
Modul inklusive Integration in das TIA Portal und
Aspekte des Software Engineerings**

Bachelorarbeit im Studiengang Informatik

vorgelegt von

Jan-Eric Gedicke

Matrikelnummer 3643446

Erstgutachter: Prof. Dr. Meitner

Betreuer: Conrad, Maximilian

© 2025

Inhaltsverzeichnis

1	Einleitung	1
1.1	Problemstellung und Motivation	1
1.2	Zielsetzung der Arbeit	1
1.3	Methodisches Vorgehen und Struktur der Arbeit	2
2	Grundlagen	3
2.1	SPS-Technologie und Module im industriellen Umfeld / Überblick über SIMATIC S7 bei Thomas oder Andreas	3
2.2	Überblick über das FM350-2 Modul	6
2.3	Eigenschaften und Aufbau des TM FAST von Andreas (wichtige sachen für software architektur)	6
2.4	Maybe	7
3	Analyse der Ablösung des FM350-2 durch das TM FAST Modul	8
3.1	Funktionsweise und Limitierungen des FM350-2	8
3.2	Einsatzgebiete der FM 350-2	10
3.3	Vorteile und neue Möglichkeiten durch das TM FAST Modul	11
3.4	Anforderungen an die Neuentwicklung	11
4	Entwicklung des VHDL-basierten Applikationsbeispiels	13
4.1	Konzept und Architektur des neuen Applikationsdesigns	13
4.2	Umsetzung der Funktionalität in VHDL	14
4.3	Test und Validierung der VHDL-Implementierung	14
4.4	Schnittstelle mit dem TIA Portal	14
4.5	Vergleich mit der bisherigen Lösung	14
5	Fazit und Ausblick	15
5.1	Zusammenfassung der Ergebnisse	15
5.2	Kritische Reflexion und Herausforderungen	15
5.3	Potenzielle Weiterentwicklungen und zukünftige Anwendungen . . .	15

<i>Inhaltsverzeichnis</i>	iii
6 Plan	17
7 Fragen	20
8 Wichtig	23
9 TODO	25
10 Tests	27
11 Passwörter	29

1 Einleitung

1.1 Problemstellung und Motivation

Die Produktserie S7-300, insbesondere das FM350-2 Modul, sind nicht mehr in der aktiven Vermarktung und dadurch nicht mehr bestellbar. Das FM350-2 Modul bietet 8 Kanäle für hochkanalige Dosieranwendungen und ist nur noch als Ersatzteil verfügbar. Ein alternatives Lösungskonzept ist der Einsatz des TM FAST Moduls, eines freiprogrammierbaren Hochgeschwindigkeits-Moduls, welches eine erweiterte Kanalanzahl von 10 bis 12 Kanälen ermöglicht. Ziel dieser Bachelorarbeit ist die Entwicklung eines VHDL-basierten Applikationsbeispiels zur Demonstration der Einsatzmöglichkeiten des TM FAST Moduls in hochkanaligen Dosieranwendungen. Gleichzeitig soll die Integration in das TIA Portal erfolgen, um eine reibungslose Nutzung in industriellen Automatisierungsumgebungen zu gewährleisten.

Als dualer Student bei Siemens verfüge ich bereits über Erfahrung in der industriellen Automatisierungstechnik. Diese Arbeit bietet mir die Gelegenheit, mein Wissen in den Bereichen Embedded Systems, Industrieautomation und Software Engineering weiter zu vertiefen. Ich freue mich darauf, dieses praxisnahe Thema in Kooperation mit Siemens zu bearbeiten und einen wertvollen Beitrag zur Weiterentwicklung industrieller Automatisierungslösungen zu leisten.

1.2 Zielsetzung der Arbeit

Ziel dieser Bachelorarbeit ist die Entwicklung eines VHDL-basierten Applikationsbeispiels zur Ablösung des FM350-2 Moduls durch das TM FAST Modul. Dabei soll die Kanalanzahl von 8 auf 10 oder 12 erhöht werden, um eine flexiblere Nutzung in hochkanaligen Dosieranwendungen zu ermöglichen. Ein wesentlicher

Bestandteil der Arbeit ist zudem die Integration des entwickelten Applikationsbeispiels in das TIA Portal, um eine nahtlose Einbindung in bestehende Automatisierungslösungen zu gewährleisten. Neben der Implementierung des VHDL-Codes werden bewährte Methoden des Software Engineerings angewendet, um die Code-Qualität und Skalierbarkeit sicherzustellen. Schließlich soll eine umfassende Dokumentation erstellt werden, die eine detaillierte Beschreibung der Nutzung und Inbetriebnahme des Moduls umfasst.

1.3 Methodisches Vorgehen und Struktur der Arbeit

Zur Umsetzung der Ziele werden zunächst die bestehenden Funktionalitäten des FM350-2 Moduls analysiert und an den aktuellen Erfordernissen der Kunden gespiegelt. Dabei werden sowohl die Hardware- als auch die Software-Architektur untersucht und relevante Funktionen für die Dosieranwendung identifiziert. Anschließend erfolgt die Entwicklung des VHDL-basierten Applikationsbeispiels, wobei die ausgewählten Funktionen des FM350-2 Moduls in VHDL umgesetzt und die Kanalanzahl auf 10 oder 12 erweitert wird. Danach wird die Integration in das TIA Portal vorgenommen, indem das TM FAST Modul an das Automatisierungssystem angebunden und geeignete Schnittstellen sowie Datenstrukturen entwickelt werden. Qualitätssicherungsmaßnahmen spielen ebenfalls eine zentrale Rolle. Hierbei werden bewährte Entwicklungswerkzeuge wie Intel Quartus Prime und das TIA Portal genutzt, Teststrategien zur Funktionssicherung implementiert und eine modulare Softwarearchitektur entworfen. Abschließend wird die Dokumentation erstellt, die eine umfassende Beschreibung der Implementierung und Nutzung des Moduls sowie Anwendungsbeispiele und Testberichte enthält. Die Ergebnisse werden in einer Abschlusspräsentation vorgestellt.

2 Grundlagen

2.1 SPS-Technologie und Module im industriellen Umfeld / Überblick über SIMATIC S7 bei Thomas oder Andreas

Ich:

Speicherprogrammierbare Steuerungen (SPS) sind durch ihre freie Programmierbarkeit, vielseitig einsetzbar und dadurch ein essenzieller Bestandteil der industriellen Automatisierungstechnik. Sie werden zum steuern und regeln von Ventilen, Antrieben, Maschinen, Pumpen und Sensoren verwendet.

ChatGPT: Speicherprogrammierbare Steuerungen (SPS) sind ein essenzieller Bestandteil der industriellen Automatisierungstechnik. In diesem Abschnitt werden die grundlegenden Prinzipien einer SPS sowie deren Aufbau und Funktionsweise erläutert. Zudem werden verschiedene Modularten vorgestellt, darunter digitale und analoge Ein-/Ausgabemodule, Zählermodule sowie Kommunikationsmodule. Ein weiteres Thema ist die Anbindung von SPS-Systemen an industrielle Netzwerke mittels gängiger Protokolle wie PROFINET, PROFIBUS und Modbus.

Thomas: Speicherprogrammierbare Steuerungen – im Englischen „Programmable Logic Controllers“ (PLCs) – wie die SIMATIC S7-Serie von Siemens steuern und regeln unter anderem Maschinen, Antriebe, Ventile, Pumpen und Sensoren. Diese Steuerungen sind frei programmierbar und dadurch universell einsetzbar [10, S. 146]. Im Folgenden wird ein Überblick über den Aufbau, die Funktionsweise und die Programmierung eines SIMATIC S7-Systems gegeben.

2.1.1 aufbau eines simatic s7

Ein SIMATIC S7-System ist modular aufgebaut und besteht aus mehreren Komponenten. Im Folgenden werden die Hauptkomponenten und ihre Funktionen erläutert. **Controller (CPU)** Die CPU der Steuerung wird auch als Controller bezeichnet und bildet die zentrale Verarbeitungseinheit des Systems. Die Hauptaufgabe der CPU ist es, das Anwenderprogramm auszuführen, das vom Nutzer entwickelt wurde, um spezifische Steuerungs- und Regelungsaufgaben zu übernehmen [11, S. 25]. Die CPU verarbeitet Signale von Ein- und Ausgabemodulen, führt Berechnungen durch und gibt Steuerbefehle an angeschlossene Geräte, wie Sensoren und Aktoren, aus. Controller der SIMATIC S7 Serie verfügen über integrierte Kommunikationsschnittstellen, wie Ethernet/PROFINET oder PROFIBUS, die den Datenaustausch mit anderen Steuerungen, Bediengeräten oder übergeordneten Systemen (z. B. SCADA oder MES) ermöglichen [12, S. 14]. **Weitere Baugruppen** Um eine Verbindung zu einer Maschine oder Anlage herzustellen, werden Signalbaugruppen verwendet, welche es als Ein- bzw. Ausgabebaugruppen sowohl für Digitale als auch analoge Signale gibt wie Spannungen oder Ströme gibt. Technologiebaugruppen sind Signalverarbeitende Baugruppen, welche eingehende Signale aufbereiten und entweder direkt an den Prozess oder zur weiteren Verarbeitung an die CPU geben. Ein möglicher Einsatzzweck ist z.B. das zählen von Impulsen, wofür die CPU meist zu langsam ist. Zur Erweiterung der Kommunikationsfähigkeit der CPU bezüglich Protokollen oder Funktionen kommen Kommunikationsbaugruppen zum Einsatz [11, S. 25].

2.1.2 Funktionsweise einer Speicherprogrammierbaren Steuerung

Grundsätzlich arbeitet eine Speicherprogrammierbare Steuerung nach dem Eingabe-Verarbeitung-Ausgabe-Prinzip (EVA). Der damit verbunden Prozess erfolgt zyklisch und lässt sich wie folgt darstellen: Über Eingangsbaugruppen werden zunächst die Zustände von angeschlossener Sensorik erfasst. Diese Eingangssignale werden anschließend in einem Prozessabbild gespeichert, welches eine Momentaufnahme der aktuellen Signalzustände darstellt. Änderungen der Signale, die während eines laufenden Zyklus auftreten, werden somit ignoriert [10, S. 147], [13, S. 39]. Das Prozessabbild dient nun als Datengrundlage für die Verarbeitung durch das Anwenderprogramm. Das Anwenderprogramm ist modular aufgebaut

und arbeitet mit sogenannten Bausteinen. Das Hauptprogramm wird durch den Organisationsbaustein1 (OB1) repräsentiert, welcher aus konkreten Anweisungen aber auch Funktions- oder Unterprogrammaufrufen besteht. Die Abarbeitung des OB1 entspricht einem Verarbeitungszyklus, dessen Dauer üblicherweise im zweistelligen Millisekunden Bereich liegt [13, S. 39]. Nach der Verarbeitung des Anwenderprogramms erstellt die SPS ein neues Prozessabbild, das die Ergebnisse der Programmverarbeitung enthält. Dieses Abbild wird schließlich an die Ausgangsbaugruppen übertragen, woraufhin die angeschlossenen Aktoren angesteuert werden. Mit diesem Schritt ist der Zyklus abgeschlossen, und die SPS beginnt unmittelbar mit der Erfassung der neuen Eingangs-Signalzustände für den nächsten Zyklus [10, S. 147], [13, S. 39].

2.1.3 Programmierung einer PLC mit TIA-Portal

ChatGPT: Das TIA Portal ist eine zentrale Entwicklungsumgebung für SPS-Programmierung und Systemintegration. Hier werden der Aufbau und die Struktur des TIA Portals beschrieben, einschließlich der unterstützten Programmiersprachen wie KOP (Kontaktplan), FUP (Funktionsplan) und SCL (Structured Control Language). Weiterhin wird die Integration von Modulen wie FM 350-2 oder TM FAST erläutert. Schließlich werden Diagnosemöglichkeiten und Optimierungsstrategien für SPS-Programme im TIA Portal vorgestellt.

Das Totally Integrated Automation Portal (TIA Portal) ist ein Framework für die Projektierung, Programmierung und Inbetriebnahme von SIMATIC S7-Steuerungen und deckt unter anderem Hardware-Konfiguration, Softwareentwicklung und Fehlerdiagnose ab [14, S. 11]. Die Programmierung folgt dabei der internationalen Norm IEC 61131-3, die den Standard für die Softwareentwicklung in speicherprogrammierbaren Steuerungen (SPS) definiert. Diese Norm legt die Struktur und die Programmiersprachen fest, die in Automatisierungsprojekten eingesetzt werden können, um eine einheitliche und herstellerübergreifende Programmierbarkeit zu gewährleisten. Nach IEC 61131-3 werden fünf standardisierte Programmiersprachen für SPS definiert: KOP (Kontaktplan): Eine grafische Sprache, die sich an elektrischen Schaltplänen orientiert. FUP (Funktionsplan): Eine grafische Sprache zur Darstellung von Logik und Funktionen. AWL (Anweisungsliste): Eine textbasierte Sprache, die ähnlich wie Assembler arbeitet. SCL (Structured Control

Language): Eine textbasierte, hochsprachliche Sprache ähnlich zu Pascal. AS (Ablaufsprache): Eine grafische Sprache zur Darstellung von Ablaufsteuerungen [10, S. 153]. Für die Programmierung im Rahmen der Bachelorarbeit sind insbesondere die Sprachen FUP und SCL relevant. **Funktionsplan (FUP)** UP ist eine grafische Programmiersprache. Sie basiert auf der Darstellung von Funktionsbausteinen, die durch Verbindungen miteinander verknüpft werden. Ein typisches Beispiel in FUP ist die Implementierung von UND-, ODER- oder NICHT-Verknüpfungen, bei denen Ein- und Ausgangssignale miteinander verbunden werden. Die Funktionsbausteine enthalten logische Funktionen, mathematische Operationen oder auch spezifische Steuerungsaufgaben, die als grafische Symbole dargestellt sind [15, S. 91 f.]. **Structured Control Language (SCL)** SCL ist eine textbasierte Programmiersprache, die der Hochsprache Pascal ähnelt. Sie ist besonders für komplexere Algorithmen und mathematische Berechnungen geeignet. SCL wird in der IEC 61131-3 als Strukturierter Text bezeichnet und unterstützt eine strukturierte Programmierung mit Schleifen, Verzweigungen und Funktionsaufrufen [10, S. 153].

2.2 Überblick über das FM350-2 Modul

ChatGPT: Das FM 350-2 Modul ist ein spezielles Zähl- und Messmodul für SPS-Systeme. Hier werden die technischen Spezifikationen und die grundlegende Funktionsweise erläutert. Typische Anwendungen, wie die Messung von Drehzahlen oder das Zählen von Impulsen, werden beschrieben. Darüber hinaus werden die Einbindung in SPS-Systeme, die verfügbaren Steuer- und Leseoperationen sowie die Limitierungen des Moduls thematisiert.

2.3 Eigenschaften und Aufbau des TM FAST von Andreas (wichtige sachen für software architektur)

ChatGPT: TM FAST Modul ist eine FPGA. FPGAs (Field Programmable Gate Arrays) bieten eine leistungsstarke Alternative zu klassischen SPS-Modulen. Dieser Abschnitt behandelt die Grundlagen der VHDL-Programmierung und die

Architektur von FPGAs. Ein besonderer Fokus liegt auf den Unterschieden zwischen FPGAs, Mikrocontrollern und herkömmlichen SPS-Modulen sowie deren jeweiligen Vor- und Nachteilen in der Automatisierungstechnik. Abschließend werden praxisnahe Beispiele für die Implementierung von Zähl- und Messfunktionen mit VHDL vorgestellt.

ChatGPT: Das TM FAST Modul stellt eine Weiterentwicklung gegenüber dem FM 350-2 dar und ist für hochdynamische Anwendungen konzipiert. In diesem Abschnitt werden die technologischen Unterschiede sowie die Vorteile des TM FAST Moduls in Bezug auf Echtzeitfähigkeit und Signalverarbeitung betrachtet. Außerdem wird die Kompatibilität des Moduls mit bestehenden SPS-Systemen analysiert.

2.4 Maybe

-Automatisierungspyramide -Echtzeitsysteme? übergang zu was ich mache (ist das wichtig? wahrscheinlich nicht) -verwendete Software:Questa, Quartus, TIA ... (Bei Umsetzung mit 3 Sätzen)

3 Analyse der Ablösung des FM350-2 durch das TM FAST Modul

3.1 Funktionsweise und Limitierungen des FM350-2

ChatGPT: Die FM 350-2 ist ein Zähl- und Messmodul, das speziell für schnelle Zählprozesse und Frequenzmessungen entwickelt wurde. In diesem Abschnitt werden die wesentlichen Funktionen, deren Anwendung sowie die Einschränkungen des Moduls beschrieben.

3.1.1 Grundlegende Funktionen der FM 350-2

Die FM 350-2 bietet eine Vielzahl von Funktionen zur Steuerung und Überwachung von Zählprozessen. Hierzu gehören unter anderem das Initialisieren der Zähler, das Steuern von Digitalausgängen sowie das Laden und Auslesen von Zählwerten.

3.1.1.1 Steuerung der Baugruppe mit FC CNT2_CTR

(FC2)

Die Funktion extttFC CNT2_CTR ermöglicht die Steuerung der Digitalausgänge der FM 350-2 sowie die Verwaltung der Software-Tore. Zudem können Rückmeldesignale ausgelesen werden.

Hauptfunktionen:

- Initialisierung des Zähler-DBs
- Auslesen der Rückmeldesignale und Speicherung in der Struktur CHECKBACK_SIGNALS

- Übertragen der Steuersignale aus der Struktur CONTROL_SIGNALS zur FM 350-2

Die Funktion muss zyklisch (z. B. im OB1 oder im Weckalarm OB35 der S7-300) aufgerufen werden, um eine kontinuierliche Überwachung und Steuerung zu gewährleisten.

3.1.1.2 Laden von Zählerständen, Grenz- und Vergleichswerten (extttFC3 / extttFB3)

Mit der Funktion extttFC CNT2_WR oder dem Funktionsbaustein extttFB CNT2WRPN können Zählerstände und Vergleichswerte neu geladen werden. Diese Funktion sollte nur verwendet werden, wenn im laufenden Betrieb neue Werte in das Modul eingespielt werden müssen.

3.1.1.3 Auslesen von Zähl- und Messwerten (extttFC4 / extttFB4)

Die extttFC CNT2_RD / extttFB CNT2RDPN Funktion dient zum zyklischen Auslesen der aktuellen Zählwerte. Falls keine Leseaufträge benötigt werden, kann auf diese Funktion verzichtet werden, um die Prozesslast zu minimieren.

3.1.1.4 Diagnosedaten lesen (FC5)

Im Falle eines Diagnosealarms ermöglicht die Funktion FC DIAG_RD das Laden der Diagnosealarmdaten in den Zähler-DB, um Fehlerursachen zu analysieren und entsprechende Maßnahmen einzuleiten.

3.1.2 Einschränkungen der FM 350-2

Trotz ihrer vielseitigen Einsatzmöglichkeiten weist die FM 350-2 einige Limitierungen auf:

- Begrenzte Flexibilität in der Anpassung an komplexe Steuerungsaufgaben
- Kein FPGA-basierter Aufbau, wodurch individuelle Anpassungen nicht möglich sind

- Begrenzte Echtzeitfähigkeit im Vergleich zu modernen Zählmodulen

3.2 Einsatzgebiete der FM 350-2

3.2.1 Haupteinsatzgebiete

Die FM 350-2 wird hauptsächlich dort eingesetzt, wo Signale gezählt und schnelle Reaktionen auf vorgegebene Zählerstände erforderlich sind. Besonders relevant ist sie für Anwendungen in der Fertigungs- und Automatisierungstechnik.

3.2.2 Typische Anwendungen

Typische Einsatzbereiche sind:

- Verpackungsanlagen
- Sortieranlagen
- Dosieranlagen
- Drehzahlregelungen und Überwachung von Gasturbinen

3.2.3 Anwendungsbeispiel: Kartonabfüllung

Ein typisches Beispiel für die Nutzung der FM 350-2 ist die Abfüllung von Teilen in Kartons:

- Kanal 0 zählt die Teile und steuert das Abfüllventil.
- Kanal 1 steuert den Transport der Kartons und erfasst die Anzahl der gefüllten Kartons.
- Das System stellt sicher, dass exakt die vorgegebene Anzahl an Teilen pro Karton abgefüllt wird.

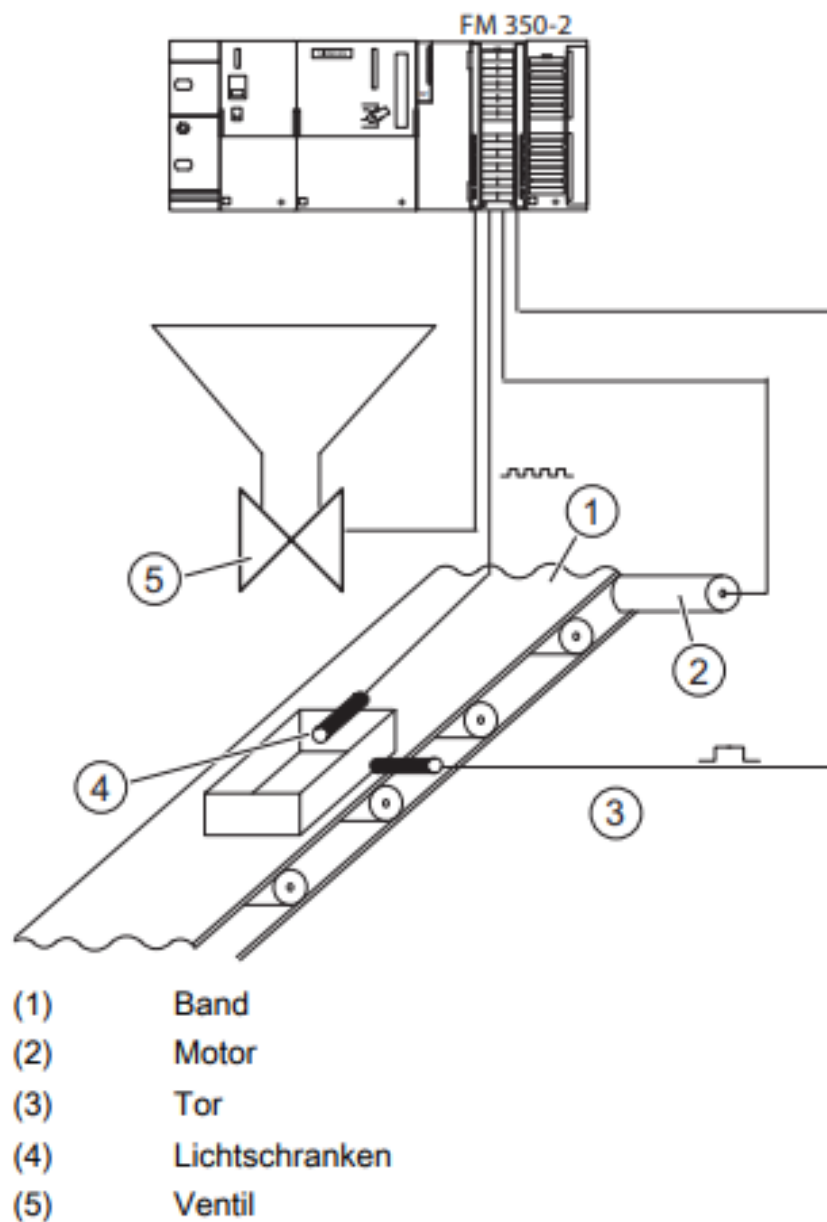


Abbildung 3.1: Beispiel für den Einsatz einer FM 350-2 in der S7-300 [7]

3.3 Vorteile und neue Möglichkeiten durch das TM FAST Modul

3.4 Anforderungen an die Neuentwicklung

Kriterium	TM FAST	FM 350-2
Unterschiede		
Architektur	FPGA-basiert, flexible und programmierbare Hardwareplattform	Reines Zählermodul ohne FPGA-Technologie
Funktionalität	Kann komplexe Prozesssteuerungs- und Überwachungsaufgaben übernehmen	Spezialisiert auf Zählen, Frequenz- und Drehzahlmessung
Prozessintegration	Kann direkt in den Produktionsprozess eingreifen und Prozessalarmlöser auslösen	Liefert Zähler- und Messwerte, die von der übergeordneten Steuerung weiterverarbeitet werden
Flexibilität	Hohe Flexibilität und Anpassungsfähigkeit durch FPGA-Architektur	Festgelegte Funktionalität, die nicht ohne Weiteres erweitert werden kann
Gemeinsamkeiten		
Zählfunktionen	Beide Module bieten Zählfunktionen mit hoher Auflösung (z.B. 32-Bit Zähltiefe)	
Frequenz- und Drehzahlmessung	Beide Module unterstützen die Messung von Frequenzen und Drehzahlen	
Einbindung in Steuerungsarchitektur	Beide Module werden in die übergeordnete Steuerungsarchitektur integriert	
Industrielle Anwendungen	Beide Module finden Einsatz in ähnlichen Industriebereichen wie Verpackung, Sortierung, Dosierung etc.	

Tabelle 3.1: Vergleich zwischen TM FAST und FM 350-2

4 Entwicklung des VHDL-basierten Applikationsbeispiels

Hier ist maybe wichtig Aspekte des Software Engineerings, Best Practices, Modularisierung und Wiederverwendbarkeit, Teststrategien und Debugging-Techniken

4.1 Konzept und Architektur des neuen Applikationsdesigns

Byte Zuweisung für FB_IF (32 Byte)

- 3 Byte für Kanalobergrenze = 4095 (Standartventil ist 1 Impuls/ 1 ml -> 4l abfüllung) => bis zu 10 Kanäle möglich
- 4 Byte für Kanalobergrenze = 65535 (65l abfüllung) => nur 8 Kanäle möglich

Andere Ideen: Vielleicht auf TMFASTUserWriteREC ausweichen? (Bis zu 128 Byte) asynchron (was ist wenn daten mitten im ablauf geändert werden?)

Algorithmus, der im ersten Zyklus die Kanalobergrenzen 1–4 von der CPU an das TMFast sendet, im zweiten Zyklus 5–8 usw. ist synchron und eignet sich besser die Steuerschnittstelle, da über die im Betrieb auch taktsynchron neue Werte geschrieben werden können. Der Nachteil des Write-Records ist die Asynchronität zum Zyklus.

Zähler -> Unter- und Obergrenze einstellbar

CPU Stop beachten:

4.2 Umsetzung der Funktionalität in VHDL

-Diagramme einfügen, beschreiben und analysieren

4.3 Test und Validierung der VHDL-Implementierung

-Questa simuliert Eingänge und prüft die Ausgänge

4.4 Schnittstelle mit dem TIA Portal

-Diagramm für Schnittstelle und Funktion der TIA Anwendung

4.5 Vergleich mit der bisherigen Lösung

-Auf abschnitte in der Analyse eingehen

5 Fazit und Ausblick

5.1 Zusammenfassung der Ergebnisse

5.2 Kritische Reflexion und Herausforderungen

5.3 Potenzielle Weiterentwicklungen und zukünftige Anwendungen

Literaturverzeichnis

- [1] Ernst Klett Verlag (2004): *Infoblatt Siemens AG*. Verfügbar unter: <https://www.klett.de/alias/1036900>. Zugriff: 5. Januar 2025.
- [2] Merkur (2022): *Siemens – Geschichte, Aktie und Tätigkeitsfelder*. Verfügbar unter: <https://www.merkur.de/wirtschaft/siemens-geschichte-aktie-und-taetigkeitsfelder-91268844.html>. Zugriff: 19. Januar 2024.
- [3] Siemens, 2023, zitiert nach de.statista.com: *Umsatz von Siemens SE seit 2005*. Verfügbar unter: <https://de-statista-com.thn.idm.oclc.org/statistik/daten/studie/73827/umfrage/umsatz-von-siemens-seit-2005/>. Zugriff: 5. Januar 2025.
- [4] ResearchGate: Verfügbar unter: https://www.researchgate.net/figure/SA-88s-physical-model-and-an-example-production-allocation_fig2_308384237/. Zugriff: 11. Januar 2025.
- [5] Siemens (2022): *Pflichtenheft-SmartFactory*. Internes Dokument.
- [6] Liam Bee (2022): *PLC and HMI development with Siemens TIA Portal: develop PLC and HMI programs using standard methods and structured approaches with TIA Portal V17*. Packt Publishing, Limited.
- [7] Siemens AG (2011): *Zählerbaugruppe FM 350-2 Gerätehandbuch*. Verfügbar unter: https://cache.industry.siemens.com/dl/files/178/1105178/att_20226/v1/s7300_fm350_2_operating_instructions_de_de-DE.pdf. Zugriff 11. März 2025.
- [8] Siemens AG (2023): *TMFast Programmeier Handbuch*. Verfügbar unter: https://cache.industry.siemens.com/dl/files/088/109816088/att_1144842/v2/s71500_tm_fast_programming_manual_de-DE_de-DE.pdf. Zugriff 12. März 2025.

6 Plan

Hier sind einige Schritte, die Ihnen dabei helfen können, eine VHDL-Applikation für das TM FAST-Modul zu entwickeln, um das alte FM350-2-Modul zu ersetzen:

Anforderungsanalyse

- Analysieren Sie sorgfältig die Funktionalität und Schnittstellen des FM350-2-Moduls.
- Identifizieren Sie die Kernfunktionen, die das neue TM FAST-Modul erfüllen muss.
- Definieren Sie klar die Eingangs- und Ausgangssignale, die Kommunikationsprotokolle und andere relevante Spezifikationen.

Architekturentwurf

- Entwerfen Sie eine modulare Architektur für Ihre VHDL-Applikation.
- Unterteilen Sie die Funktionalität in logische Blöcke wie Steuerungslogik, Kommunikationsschnittstellen, Datenpfade usw.
- Legen Sie die Schnittstellen zwischen den einzelnen Modulen fest.

VHDL-Entwicklung

- Beginnen Sie mit der Implementierung der einzelnen VHDL-Module basierend auf Ihrem Architekturentwurf.
- Achten Sie auf eine strukturierte und modulare Codeerstellung, um die Wartbarkeit und Erweiterbarkeit zu erleichtern.
- Verwenden Sie VHDL-Sprachkonstrukte wie Prozesse, Signale, Typen und Pakete, um Ihre Applikation übersichtlich und effizient zu gestalten.

Simulation und Test

- Erstellen Sie umfangreiche Testbänke, um Ihre VHDL-Applikation gründlich zu überprüfen.
- Führen Sie Simulationen durch, um das korrekte Verhalten der einzelnen Module und der Gesamtapplikation zu verifizieren.
- Testen Sie die Applikation schrittweise, um mögliche Fehler frühzeitig zu erkennen und zu beheben.

Integration und Validierung

- Integrieren Sie Ihre VHDL-Applikation in die Gesamtarchitektur des TM FAST-Moduls.
- Testen Sie die Applikation in der realen Umgebung, um ihre Kompatibilität und Funktionalität zu validieren.
- Passen Sie die Applikation gegebenenfalls an, um Anforderungen und Spezifikationen des TM FAST-Moduls zu erfüllen.

Dokumentation und Versionskontrolle

- Erstellen Sie eine ausführliche Dokumentation, die die Architektur, Schnittstellen, Konfiguration und Verwendung Ihrer VHDL-Applikation beschreibt.
- Nutzen Sie ein geeignetes Versionskontrollsystem, um Änderungen an Ihrer Applikation nachzuverfolgen und zu verwalten.

Durch diese strukturierte Vorgehensweise können Sie sicherstellen, dass Ihre VHDL-Applikation für das TM FAST-Modul die Anforderungen des FM350-2-Moduls erfüllt und in die Gesamtarchitektur des TM FAST-Systems nahtlos integriert wird.

7 Fragen

Müssen Alarme umgesetzt werden? -> Müssen nicht berücksichtigt werden
Müssen Hardwareconfigs von der CPU oder von TMFast gehandhabt werden? -> hardwareconfig
der (z.B. Ventile) ist in VHDL (andere Kommunikationsprotokolle und co pro z.B.
Ventile) Wie verhalten sich die In- und Outputs?

- Sollen alle Eingänge Zähl- und Richtungseingang sein oder Aufteilung zwischen den beiden? Nur zähler
-

wie funktionieren ventile überhaupt?

bit an die cpu senden

nachlauf bei inkrement zum abfüllen

cpu macht - tm fast macht ventil zu

auf ist 1 zu ist low für ventil

abfüllende menge muss übergeben werden

vhdl auf hoher ebene darstellen (als baustein)

synchron oder asynchron (für zähler oder generell) -> Alles Synchron (TM Fast arbeitet (einstellbar mit 50mHz) schneller als ein ventil) -> dadurch zählt es schneller als inkremente des ventils

clk wird bei änderung aktiviert (High->Low / Low->High)

steuerung des ventils über PLC oder TMFAST? Erstmal Plc mit bool senden (überlauf für counter (justierung)) / TMFast erfordert Kommunikations-Protokolle
Filter?

Anwendung in mehreren oder einem Process in vhdl?

soll der timer nur zählen, wenn die flanke positiv wird? oder auch wenn sie negativ wird? -> nur bei positiv

Allgemein: Schriftgröße und Seitenränder? Zitieren von Handbüchern (Graue Literatur) und ChatGPT? Mit Disclaimer

ES IST KEIN PRAXISBERICHT (KEIN PROZESS BESCHREIBEN) / EVTL: VORNACHTEILANALYSE

Vielleicht auf WR_REC ausweichen, wenn FB_IF nicht groß genug?

Muss der Siemens Betreuer einen Bachelor haben? Was kommt auf den Betreuer zu (Organisatorisch)

Steuerung eines Ventils von TMFAST? Über DQ -> Wie und was wird dafür benötigt, reicht es ein signal zu senden?

Ventile 1. Inkrementalgeber mit zwei Spuren (A/B-Signal): Viele Durchflussmesser oder Antriebssysteme (z.B. Drehgeber) geben zwei phasenversetzte Impulssignale aus: Spur A und Spur B.

Anhand der Phasenlage erkennt die CPU die Richtung:

Wenn A vor B kommt → Vorwärts

Wenn B vor A kommt → Rückwärts

Diese Technik nennt sich Quadraturauswertung. 24 V (HTL) sind Standardventil muss aber auch andere können:

nur die Anzahl der Impulse wichtig? 12 Kanäle sind nur 6 Alternativen? RS485 für Impulse -> JA? TTL-Signals Transistor-Transistor-Logik?

Willi: A/B oder A/B/N Geber? RS422? 24V für DI und 5V für CH. Anforderungen: Gleiche Technologie? flexibel (wie umsetzen)? Nur einen Eingang (A) war bei 350-2 mit (A/B) und 24V (DI) als Eingang

Maybe:

- A -> 5V * 8 / (DI) 24V * 8 / (DIQ)? 24V * 4
- A/B -> 5V * 4 / (DI) 24V * 4 / (DIQ)? 24V * 2
- A/B/N -> 5V * 2 / (DI+DIQ?) 24V * 3

+CH0 kann nicht beides verwendet werden -> EIA-485, auch als RS-485, benutzt ein Leitungspaar, um den invertierten und einen nichtinvertierten Pegel eines 1-Bit-Datensignals zu übertragen. Am Empfänger wird aus der Differenz der beiden Spannungspegel das ursprüngliche Datensignal rekonstruiert

wo finde ich noch informationen zum software reset (ResetSW)?

8 Wichtig

Effectuated Interfaces

Control interface: control interface, direction: from CPU to TM FAST

Feedback interface: feedback interface, direction: from TM FAST to CPU

Read record: asynchronous read record data from TM FAST to CPU

Write record: asynchronous write record data from CPU to TM FAST

Anforderungen: Vergleichswertübergabe azyklische über UserWriteREC 8x 24V Geber (evtl. auf 12 Upgraden mit DIQ) CounterReset -> Setze Counter=0 CounterHold -> Ignoriere Geber-Impulse ResetSW -> Setzt MAX_VALUE=0 und Counter=0 Load -> Setzt MAX_VALUE und Counter=0

- Es gibt 12 Kanäle. Ich würde jeweils die Eingänge und Ausgänge aus der gleichen Leistungsklasse verknüpfen (DI-DQ; Umschaltbare DIQ; RS485)
- Der Sollwert, an dem abgeschaltet werden soll, wird über den Anwenderdatensatz übermittelt, Datensatz mit 32 Byte, 2 Byte Kennung und dann 14 mal der Sollwert als Integer (2 byte). Oder direkt in der Steuerschnittstelle. Der Datensatz kann im Betrieb nachgeladen werden mit allen Werten.
- Die Eingänge zählen jeweils von 0 bis zum eingestellten Sollwert und dann nahtlos wieder bei 0.
- Die Ausgänge werden mit einer Flanke in einem Bit pro Kanal in der Steuerschnittstelle gestartet und durch das Erreichen des Sollwerts wieder zurückgenommen. Es ist mit einem gemeinsamen Parameter für alle Kanäle einstellbar, ob ein Ausgang mit high aktiv ist oder mit low. Wird vor Erreichen des Sollwerts das Bit in der Steuerschnittstelle (SS) zurückgenommen wird auch der Ausgang wieder deaktiviert

- In der Rückmeldeschnittstelle wird der Zustand jedes Ausgangs angezeigt. Zusätzlich für jeden Kanal der aktuelle Zählwert oder die aktuelle Durchflussgeschwindigkeit angezeigt. Der Wert wird über ein Steuerbit in der Steuerschnittstelle ausgewählt. Der Wert für die Durchflussgeschwindigkeit könnte auf Ink/10s hochgerechnet werden, um eine entsprechende Auflösung zu erreichen. Ein bit in der SS zeigt an, welcher Wert gemeldet wird.

9 TODO

->A/B einbauen + testfall + andere testfälle (Nur A) ->Kanalbelegungen nachfragen (erstmal nur DI/DIQ) ->Grenzwertüberschreitung von enc_counter, wenn upperlimit < enc_counter mit in betracht ziehen. Reset wenn wert änderung bei upper_limit? Muss nicht, ist gewolltes Verhalten, auch nicht prüfen

->Reset über CTRL_IF (Sollte es Upper_Limit zurücksetzen?) ->UserWriteREC für die Obergrenzen einbauen (davor über CTRL_IF0(31 downto 16)/ Neue Anforderungen: Obergrenze muss nicht im Betrieb geändert werden) -> von 2 Byte auf 4 Byte durch mehr Speicher möglich ->Counterreset und Enable auf bits zuweisen (ähnlich wie reset) ->Test für overflowdetection ist useless

->UserWriteREC simulieren und Testen und mit fsm stabiler machen ->load,reset,hold logik überarbeiten(wann wird was ausgeführt und wodurch -> Muss nach einem Reset ein Load kommen um upperlimit neu zu laden (schaue beim obrgiegen simulieren)?)

Feedback gespräch -Diagramm für Ablauf -was hab ich mir gedacht -rst einbauen -Anforderungen mit aufschreiben

-Fragen dos donts ->Fault and diagnostic evaluation? -> DQ Force = 0 oder auch noch FB_IF? ->asynchronous signal WR_REC_NEW to the CLK domain ->CTRL_IF Signale auf variable zuweisen oder direkt verwenden? alias verwenden

Antwort: ->CTRL_IF für fehler quittieren, wenn overflow zu schnell toggelt ->ganzes paket ins netzwerklaufwerk des teams und diagramm und link des Product backlog ->generate für weniger logik (weniger kanäle/flexibel)

tobi berichten, dass durch test es sich etwas nach hinten verschiebt

->Review des Codes ->Quartus Compilieren und zum test TGEN (Am Ende)-> Test und angeschlossenden Enc_Enum (Gerät Geber-Simulation) (Davor evtl mit sich selbst DQ auf DI) ->Nightly Test einbauen

->wird wr_rec00 richtig gesetzt? testen

10 Tests

rückgabe über das (fb_if) feedback_interface

EncEmu encoder für impulse

schwierigkeit: di1 oder di0 zählen

RD_Rec sind azyklisch und nur selten benutzt

library test 8 counter -> name mit test cases Jürgen, ..., Kollege aus Prag

Muss ich nicht allein schreiben

Mathias fragen wenn funktionierendes Programm -> in den Nightly test muss in den nightly build

-systemlogic updaten auf v2 / DONE -zähler des sea teams verwenden / -Questa Simulation / (Questa DUT=Design under Test)

- 0000_0000_0000_0000_0000_0000_0000_0001 -> FB_IF(0)(0)<='1'; - /-> Q0.0 (FB_IF(0))

- FB_IF(1)(0) == QD4

-mathias jupiter.tfs.siemens.net für nightly build (muss ins git)

-adress register verbessern

-quellcode verwaltung?

Reaktion auf falsche Parameter Verifikation der korrekten Initialisierung des Encoders nach LOAD Grundkonfiguration und Initialisierung -> System-Reset wird ausgelöst

Verschiedene Zählmodi oder Edge-Typen?

- Zählen mit A oder A&B oder A&B&N -> nicht nötig

- Betriebsart: periodisches Zählen, einmaliges Zählen, usw. -> nicht nötig
- hoch/runterzählen, Invertieren der Hauptzählrichtung -> nicht nötig
- Test des Verhaltens beim Überlauf an Zählgrenzen (oben/unten) -> abhängig von Betriebsart -> Überlauf ist getestet/ Betriebsart ist unwichtig, da nur Hochgezählt wird (B=Low)
- Start nicht mit Wert 0 sondern Startwert -> Machbar aber muss aber nicht
- invertieren der A/B/N Eingangssignale -> ?
- Test der verschiedenen Resetquellen -> Muss

undefinds werden kopiert -> am start muss next_upper_limit initialisiert werden mit einer 0, sonst wird bei jedem load bevor WR_REC_NEW gemacht wird, next_upper_limi auf undefiend gesetzt.

11 Passwörter

PLC: Siemens123

- [Masterarbeit von Andreas Kick](#)
- [Bachelorarbeit von Thomas Weisel](#)
- [Prof. Meitner Zoom Link](#)
- [Diagramme](#)